

Week 08: From API Calls to Agents

First Principles of the ReAct Pattern & Autonomous AI

WEEK	08	DATE	07 MAR 2026	FOCUS	REACT PATTERN & AGENTS
------	----	------	-------------	-------	------------------------

Demystifying how LLMs transition from static chatbots to goal-driven autonomous agents capable of reasoning, acting on the world through tools, and iterating until the job is done.

Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Colab Notebook	Google Colab (Local Copy: Codes/class_6.ipynb)
 Excalidraw Board	Interactive Whiteboard
 Assignment	Google Colab

Table of Contents

Section	Topic	Description
	Week 07 Recap	RoPE, GQA, SwiGLU, Pre-Norm — the internals that make agents fast enough
1	The Evolution to Agents	From Autocomplete → Chat → Goal-Driven Autonomy
2	Defining the AI Agent	The LLM + Tools + Loop triad
3	Your First API Call	OpenRouter setup, temperature, and the memory illusion
4	Function Calling	How models REQUEST tools via structured JSON
5	The ReAct Loop	THOUGHT → ACTION → OBSERVATION → repeat
6	Building a ReAct Agent	~60 lines of Python that replicate Claude Code's core
7	Tracing & Analyzing	Token growth, the context engineering problem
8	The Code Agent	File I/O + Python execution = an autonomous developer
9	Native Function Calling	API-level tool_calls — no regex needed

Section	Topic	Description
10	The "Gotchas"	Memory, hallucinations, infinite loops, tk/s
	Homework / Challenges	4 exercises to solidify your understanding

Quick Recap: Week 07 Bridge (Transformer Internals)

Before diving into agents, let's solidify the core mechanics from the **MiniLLM** we built in Week 07. These architectural choices are what make modern LLMs (like LLaMA and Claude) **fast enough** to serve as real-time agents.

The Processing Pipeline

- Text → Tokens (Subword, BPE, or Character) → Token ID → Token Embedding (e.g., dim=768).
- Example: rishabh → rish + abh → [1221, 456]
- Add **Positional Embeddings** (RoPE) and pass into the **Transformer Block**.

Autoregressive Training & Causal Masking

The model learns to predict the next token given all previous context.

- Input:** 'they enj'
- Target:** 'hey enjo' (Input shifted right by one)

To prevent the model from "cheating" and looking at the future, we apply a **Causal Mask** (a triangular matrix of **-inf**):

```
mask = torch.triu(torch.ones(seq, seq, device=x.device), diagonal=1).bool()
```

The Four Modern Upgrades (Classic → Modern)

Component	Classic (2017)	Modern (2023+)	Why It Matters for Agents
Normalization	LayerNorm (Post-Norm)	RMSNorm (Pre-Norm)	Stable gradients at 100+ layers → bigger, smarter models
Feed-Forward	ReLU	SwiGLU	Learned gating → richer representations → better reasoning
Position	Sinusoidal (Absolute)	RoPE (Rotary)	Relative distance → generalizes to long agent conversations
Attention	Multi-Head (MHA)	Grouped Query (GQA)	4-8x KV Cache reduction → faster tk/s → responsive agents

► 🔍 Click to expand detailed explanations

RMSNorm vs. LayerNorm

- **LayerNorm:** $(x - \text{mean}) / \text{std} * \gamma + \beta$ (4 operations, 2 learnable params)
- **RMSNorm:** $x / \text{RMS}(x) * \gamma$ (2 operations, 1 learnable param)
- **Takeaway:** RMSNorm abandons mean-centering and the bias term entirely — identical stability, strictly less compute.

SwiGLU (The Smarter Gatekeeper)

Replaces $\text{ReLU}(x) = \max(0, x)$. Instead of brutally killing negative values, SwiGLU splits data into two paths:

1. **Gate Path:** $\text{SiLU}()$ — a smooth probability dial.
2. **Signal Path:** Linear information.
3. Element-wise multiply $\rightarrow$ the network learns *how much* signal to let through.

RoPE (Rotary Positional Embeddings)

- **Problem:** Absolute embeddings make "cat" get completely different position info at position 0 vs position 100.
- **Solution:** Treat dimensions as pairs and *rotate* Q and K vectors by position-proportional angles.
- **Result:** When computing $Q \cdot K^T$, absolute rotation cancels out $\rightarrow$ only **relative distance** remains.

GQA (The Camera Analogy)

- **MHA:** Every reporter (Q head) gets their own camera crew (K, V heads). *Great quality, huge VRAM.*
- **MQA:** All reporters share ONE camera crew. *Low memory, notable quality drop.*
- **GQA:** Reporters clustered into groups, each sharing a camera crew. *The golden mean.*

1 The Evolution to Agents

2025-2026 is the year of AI Agents. The landscape has shifted from "tools that help you code" to "tools that code for you."

The Trajectory of Coding Assistants

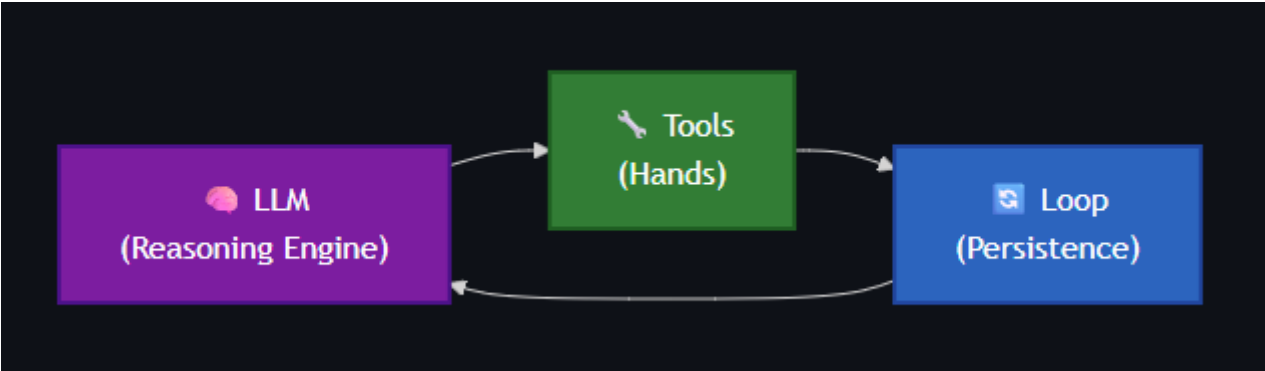
Era	Paradigm	Key Product	Developer Role
2021	AUTOCOMPLETE	GitHub Copilot	Human drives. AI suggests the next line based on local context.
2023	CHAT ASSISTANT	ChatGPT / Claude	Conversational. Users describe problems; AI provides code blocks humans copy-paste.
2025	AGENT	Claude Code / Codex	Goal-driven autonomy. Agent plans, executes, uses tools, debugs, and iterates until the objective is achieved.

[!IMPORTANT] **The key difference is the Loop.** A chat assistant gives you one response. An agent keeps going — reading files, writing code, running tests, fixing bugs — autonomously cycling until the task is complete.

2 Defining the AI Agent

What exactly transforms an LLM into an "Agent"?

The Formula: **Agent** = **LLM** + **Tools** + **Loop**



Component	Role	Critical Detail
LLM	The reasoning engine	Understands intent, plans steps, generates outputs. Has no inherent memory — the entire conversation must be re-sent every turn.
Tools	External functions	The LLM doesn't <i>possess</i> tools — it <i>requests</i> them. Your application code actually executes them.
Loop	Iterative persistence	Continuously cycles through planning → acting → observing until the goal is met or max_iterations is hit.

[!NOTE] **The Brain-in-a-Jar Analogy:** An LLM on its own is a static brain trapped in a jar — brilliant at thinking, but unable to touch anything. An agent is that same brain given hands (**Tools**) and the ability to look at what it just touched (**Loop**). The jar is open.

3 Your First API Call: Inference from First Principles

Before building agents, we need to understand the raw API layer. We use **OpenRouter** — one API key, hundreds of models, free tier available. It's **OpenAI-compatible** — same SDK, different **base_url**.

Setup

```
!pip install openai requests -q

import os, json, re, time, requests, subprocess
from openai import OpenAI
from google.colab import userdata

os.environ["OPENROUTER_API_KEY"] = userdata.get("OPENROUTER_API_KEY")

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
```

```
api_key=os.environ["OPENROUTER_API_KEY"],
)
FREE_MODEL = "openrouter/free"
```

[!TIP] To get your API key:

1. Go to openrouter.ai → create free account
2. Go to openrouter.ai/keys → create a new key
3. Add it to Colab Secrets (🔑 icon in sidebar) as `OPENROUTER_API_KEY`

⚡ Your First Inference

This is the fundamental operation. You send tokens in, you get tokens back.

```
response = client.chat.completions.create(
    model=FREE_MODEL,
    messages=[{"role": "user", "content": "What is 2 + 2? Answer in one word."}],
    temperature=0.0,
    max_tokens=50,
)

print("Response:", response.choices[0].message.content)
print(f"Tokens – in: {response.usage.prompt_tokens}, out: {response.usage.completion_tokens}")
print(f"Model used: {response.model}")
```

```
Response: Four
Tokens – in: 21, out: 4
Model used: anthropic/claude-4.5-sonnet-20250929
```

🔑 Temperature: Controlling Randomness

You already know the math: $\text{softmax}(\text{logits} / T)$. Here's the practical effect:

```
prompt = "Write a one-sentence story about a robot."

for temp in [0.0, 1.0, 2.0]:
    results = []
    for _ in range(3):
        r = client.chat.completions.create(
            model=FREE_MODEL,
            messages=[{"role": "user", "content": prompt}],
            temperature=temp, max_tokens=60,
        )
        results.append(r.choices[0].message.content.strip())
    print(f"\nTemperature = {temp}:")
```

```
for i, text in enumerate(results):
    print(f" [{i+1}] {text[:120]}")
```

Temperature = 0.0:

[1] The lonely robot tended its garden of mechanical flowers for centuries, never knowing the humans who built it were long

[2] The lonely robot tended the abandoned garden for centuries, watering flowers no human would ever see, until one day a ch

[3] The lonely robot tended its garden of mechanical flowers for centuries, never knowing the humans who built it were long

Temperature = 1.0:

[1] The lonely robot tended a garden of flowers it could never smell, watering each bloom with the same careful devotion its

[2] The lonely robot tended the last garden on Earth, watering flowers no human would ever see.

[3] The lonely robot spent centuries tending a garden on an abandoned space station, until one day a seedling finally bloome

Temperature = 2.0:

[1] A lonely robot tended a garden on an abandoned space station for three hundred years until the day a seedling finally bl

[2] When the last human died, the caretaker robot continued tending the garden for three hundred years, waiting for someone

[3] The lonely robot tended the abandoned garden for centuries, never knowing the humans who programmed it to care for the f

Temperature	Behavior	Analogy
T = 0.0	Deterministic. Nearly identical outputs every time.	An exam answer — one correct response.
T = 1.0	Diverse. Varied but coherent responses.	A creative brainstorm — many good options.
T = 2.0	Chaotic. Wild, potentially incoherent outputs.	A fever dream — unpredictable and risky.

🧠 The Model Has NO Memory

This is the single most important insight for understanding agents. The LLM has absolutely **zero memory** between API calls. "Chat memory" is an illusion created by your application **re-sending the entire conversation history** every single time.

```
conversation = [
    {"role": "system", "content": "You are a math tutor. Be concise."},
    {"role": "user", "content": "What is a derivative?"},
]
```

```
r1 = client.chat.completions.create(model=FREE_MODEL, messages=conversation,
max_tokens=150)
print(f"Turn 1 – Prompt tokens: {r1.usage.prompt_tokens}")    # 23

# Append assistant response + new user message
conversation.append({"role": "assistant", "content":
r1.choices[0].message.content})
conversation.append({"role": "user", "content": "Give me an example with x²."})

r2 = client.chat.completions.create(model=FREE_MODEL, messages=conversation,
max_tokens=150)
print(f"Turn 2 – Prompt tokens: {r2.usage.prompt_tokens}")    # 184

print(f"\n→ Prompt tokens grew from {r1.usage.prompt_tokens} to
{r2.usage.prompt_tokens}")
```

Turn 1 – Prompt tokens: 23
Turn 2 – Prompt tokens: 184

→ Prompt tokens grew from 23 to 184

[!CAUTION] **This is the core cost problem of agents.** Every iteration of an agent loop re-sends the *entire* conversation history. Over 10-50 iterations, prompt tokens compound rapidly: 266 → 355 → 432 → 507 → 580 → This is the **Context Engineering Problem** — the defining challenge of building efficient agents.

4 Function Calling: The Bridge to Agents

Function calling is the mechanism that transforms a text-only chatbot into something that can interact with the world. The model **never "calls" anything** — it generates structured JSON *requesting* a tool. **Your** code executes the function and feeds the result back.

Step 1: Define Tools with JSON Schema

We describe tools so the model knows what's available. The model doesn't HAVE these tools — it can only REQUEST them.

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather for a city. Returns temperature
and conditions.",
            "parameters": {
                "type": "object",
                "properties": {
```

```

        "city": {"type": "string", "description": "City name, e.g.
'Tokyo'"}
    },
    "required": ["city"]
}
},
{
    "type": "function",
    "function": {
        "name": "calculate",
        "description": "Evaluate a mathematical expression and return the
result.",
        "parameters": {
            "type": "object",
            "properties": {
                "expression": {"type": "string", "description": "Math
expression, e.g. '(5 + 3) * 2'"}
            },
            "required": ["expression"]
        }
    }
}
]

```

Step 2: The Model Decides to Use a Tool

When given access to tools, the model might stop generating text entirely and instead output a `tool_calls` object:

```

r = client.chat.completions.create(
    model=TOOL_MODEL,
    messages=[{"role": "user", "content": "What's the weather in Tokyo?"}],
    tools=tools, temperature=0,
)

msg = r.choices[0].message
print(f"Finish reason: {r.choices[0].finish_reason}") # tool_calls
print(f"Content: {msg.content}")                    # None
print(f"Tool calls: {msg.tool_calls}")

```

```

Finish reason: tool_calls
Content: None
→ Model wants to call: get_weather({"city": "Tokyo"})
It GENERATED JSON requesting a function. Our code must execute it.

```

Step 3: Close the Loop

Our application code intercepts the request, executes the real function, and feeds the result back:

```
# Fake tool implementations
def get_weather(city):
    fake = {"Tokyo": {"temp": "22°C", "condition": "partly cloudy"}}
    return json.dumps(fake.get(city, {"temp": "unknown", "condition": "unknown"}))

TOOL_FNS = {"get_weather": get_weather, "calculate": calculate}

# Execute the tool call
tc = msg.tool_calls[0]
fn = TOOL_FNS[tc.function.name]
result = fn(**json.loads(tc.function.arguments))

# Feed result back to model as a "tool" role message
messages = [
    {"role": "user", "content": "What's the weather in Tokyo?"},
    msg,
    {"role": "tool", "tool_call_id": tc.id, "content": result}
]

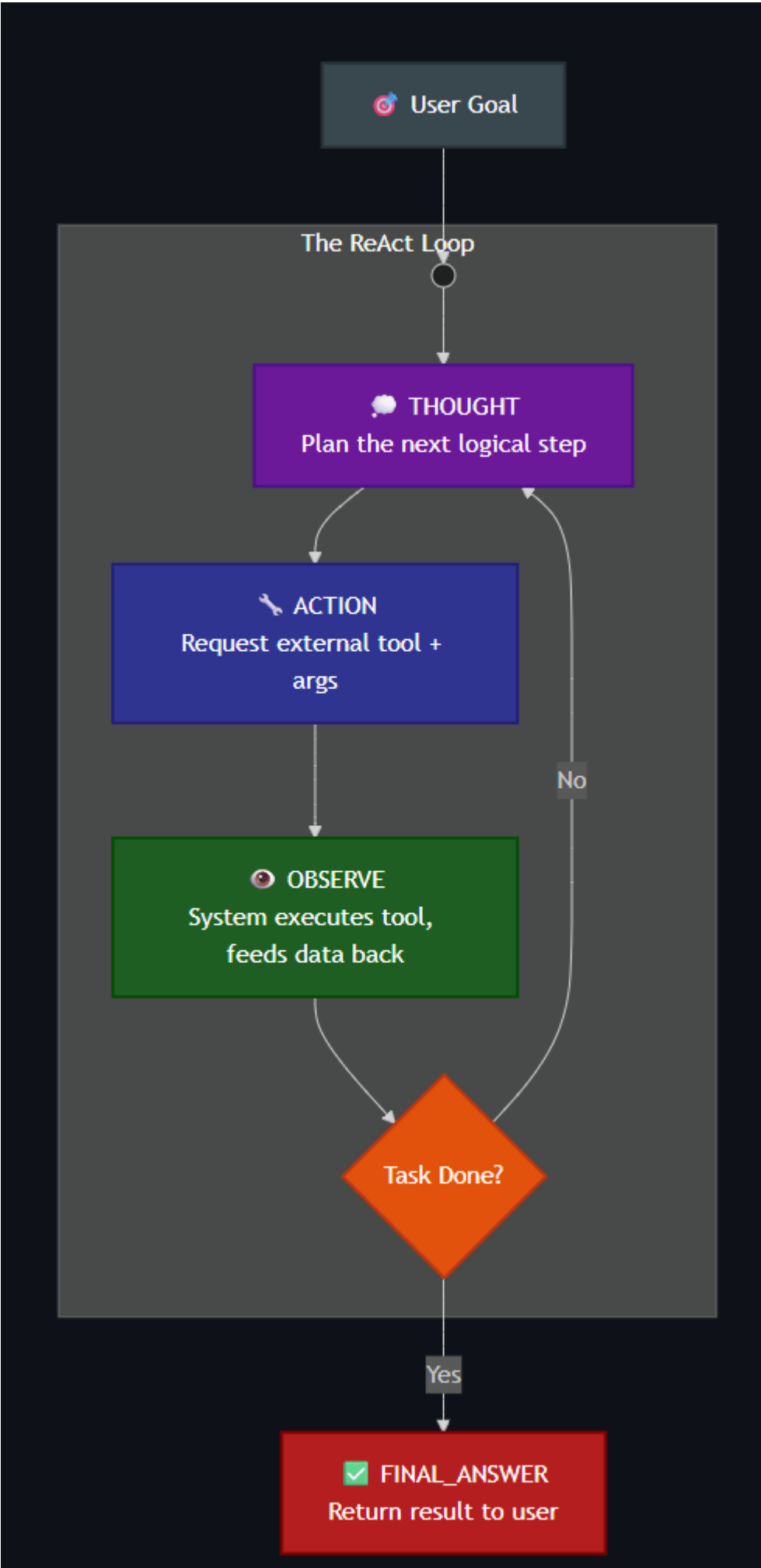
r2 = client.chat.completions.create(model=TOOL_MODEL, messages=messages,
tools=tools)
print(f"Final answer: {r2.choices[0].message.content}")
```

```
Tool result: {"temp": "22°C", "condition": "partly cloudy"}
Final answer: The weather in Tokyo is currently **22°C** with **partly cloudy**
conditions.
→ Full cycle: user → model requests tool → we run it → model answers
```

[!TIP] **The Full Cycle in one line:** User → Model requests tool → OUR code runs tool → Result back to model → Final answer

5 The ReAct Loop Demystified

The **ReAct** (Reasoning + Acting) loop is the fundamental architecture powering all modern agents. There is no hidden "magic" — it operates on a simple **"Text In → Text Out"** cycle that creates the *illusion* of sophisticated autonomous behavior.



The Three Stages

Step	Nature	What Happens
 THOUGHT	Natural Language	The model reasons about the goal, analyzes constraints, and formulates a plan.
 ACTION	Structured Output	The model emits a parseable tool request (either JSON or a regex-matchable string like ACTION: tool_name).
 OBSERVATION	Tool Result	The system halts LLM generation, runs the tool locally, and appends its real-world output back into the prompt.

(Repeat until the LLM determines: "I have enough information" and outputs **FINAL_ANSWER**.)

Why the Loop Outperforms One-Shot Prompting

One-Shot Prompting	ReAct Loop
Ask the LLM to guess the entire answer in one blind sweep	Each iteration gets a new forward pass incorporating real-world data
Model relies on stale parametric knowledge	Tools provide external truth not baked into weights
Single failure = total failure	Agent tries, fails, diagnoses, and adjusts — like a human debugging
Limited by single prompt complexity	Simple CRUD tools chain together to solve complex tasks autonomously

6 Building a ReAct Agent from Scratch

No LangChain. No CrewAI. No frameworks. Just Python + an API + a loop.

The System Prompt IS the Architecture

This prompt tells the model to alternate between THOUGHT and ACTION in a parseable format:

```
REACT_SYSTEM_PROMPT = """You are a helpful assistant that solves problems step by step using tools.

You have access to these tools:
{tool_descriptions}

## How to respond

When you need to use a tool, respond in EXACTLY this format:

THOUGHT: <your reasoning about what to do next>
ACTION: <tool_name>
ACTION_INPUT: <arguments as valid JSON>
```

When you have enough information for the final answer:

THOUGHT: <your final reasoning>

FINAL_ANSWER: <your complete answer to the user>

Rules

- Always start with THOUGHT
 - Use only ONE action per turn
 - Wait for the OBSERVATION before continuing
 - If a tool returns an error, reason about it and try a different approach
 - Be concise in your thoughts
- """

The Tool Registry

Wikipedia search hits a real API. The calculator does real math. These are the "hands" of our agent.

```
def search_wikipedia(query):
    """Search Wikipedia and return a summary."""
    url = f"https://en.wikipedia.org/api/rest_v1/page/summary/{query.replace(' ',
    '_')}"
    try:
        r = requests.get(url, timeout=10)
        if r.status_code == 200:
            data = r.json()
            return json.dumps({"title": data.get("title", ""), "summary":
data.get("extract", "")[:800]})
            return json.dumps({"error": f"Page not found for '{query}'. Try a
different term."})
        except Exception as e:
            return json.dumps({"error": str(e)})

def calculate_math(expression):
    """Safely evaluate a math expression."""
    try:
        allowed = set("0123456789+-*/.() eE")
        if not all(c in allowed for c in expression):
            return json.dumps({"error": f"Invalid characters in: {expression}"})
        result = eval(expression)
        return json.dumps({"expression": expression, "result": round(result, 6)})
    except Exception as e:
        return json.dumps({"error": str(e)})

def get_current_date():
    """Get the current date and time."""
    from datetime import datetime
    return json.dumps({"datetime": datetime.now().strftime("%Y-%m-%d %H:%M:%S")})

# Tool registry - maps names to functions and descriptions
TOOLS = {
    "search_wikipedia": {
```

```

        "fn": search_wikipedia,
        "desc": "search_wikipedia(query: str) - Search Wikipedia. Use simple topic
names."
    },
    "calculate": {
        "fn": calculate_math,
        "desc": "calculate(expression: str) - Evaluate a math expression. Example:
'(5 + 3) * 2.5'"
    },
    "get_current_date": {
        "fn": get_current_date,
        "desc": "get_current_date() - Get today's date and time. Pass empty JSON:
{}"
    },
}

```

💧 THE AGENT LOOP (~60 Lines)

This is the heart of the class. These ~60 lines implement the **exact same pattern** as Claude Code and Codex.

```

def run_agent(user_query, max_iterations=10, verbose=True):
    """A ReAct agent from scratch. No frameworks. Just an LLM + tools + a loop."""

    # Build system prompt with tool descriptions
    tool_desc = "\n".join(f"- {t['desc']}" for t in TOOLS.values())
    system = REACT_SYSTEM_PROMPT.format(tool_descriptions=tool_desc)
    messages = [{"role": "system", "content": system}, {"role": "user", "content":
user_query}]

    for i in range(max_iterations):
        # STEP 1: Ask the LLM what to do
        response = client.chat.completions.create(
            model=FREE_MODEL, messages=messages, temperature=0, max_tokens=800,
        )
        text = response.choices[0].message.content or ""
        messages.append({"role": "assistant", "content": text})

        # STEP 2: Parse the response
        thought_match = re.search(r"THOUGHT:\s*(.+?)(?=ACTION:|FINAL_ANSWER:|$)",
text, re.DOTALL)
        thought = thought_match.group(1).strip() if thought_match else ""

        # Check for FINAL_ANSWER
        if "FINAL_ANSWER:" in text:
            answer = text.split("FINAL_ANSWER:")[1].strip()
            return {"answer": answer, "iterations": i + 1}

        # Check for ACTION
        action_match = re.search(r"ACTION:\s*(\w+)", text)
        input_match = re.search(r"ACTION_INPUT:\s*(.+?)(?:\n|$)", text, re.DOTALL)

```

```

        if not action_match:
            messages.append({"role": "user", "content": "Please respond with
ACTION or FINAL_ANSWER."})
            continue

        tool_name = action_match.group(1).strip()
        raw_input = input_match.group(1).strip() if input_match else "{}"

        # STEP 3: Execute the tool
        if tool_name not in TOOLS:
            observation = json.dumps({"error": f"Unknown tool '{tool_name}'.
Available: {list(TOOLS.keys())}"})
        else:
            try:
                args = json.loads(raw_input) if raw_input.startswith("{") else
{"query": raw_input.strip("\'\'")}
                observation = TOOLS[tool_name]["fn"](**args)
            except Exception as e:
                observation = json.dumps({"error": f"Failed to call {tool_name}:
{e}"})

        # STEP 4: Feed observation back into context
        messages.append({"role": "user", "content": f"OBSERVATION:
{observation}"})

    return {"answer": "Max iterations reached.", "iterations": max_iterations}

```

Test Run: Simple Calculation

Watch the loop: THOUGHT → ACTION → OBSERVATION → FINAL_ANSWER

```
result = run_agent("What is 123*123?")
```

```

=====
👤 USER: What is 123*123?
=====

--- Iteration 1/10 ---
⚠️ Format issue – nudging...

--- Iteration 2/10 ---
💭 THOUGHT: I need to calculate 123 * 123. This is a straightforward
multiplication problem.
🔑 ACTION: calculate({"expression": "123 * 123"})
🕒 OBSERVATION: {"expression": "123 * 123", "result": 15129}

--- Iteration 3/10 ---
💭 THOUGHT: The calculation shows that 123 multiplied by 123 equals 15,129.

```

```
=====
[✓] AGENT FINISHED in 3 iteration(s)
=====
[📄] ANSWER: 15129
```

7 Tracing & Analyzing the Agent

Let's instrument the agent to understand **costs, token growth, and behavior patterns**. This is the context engineering problem — every iteration re-sends everything.

The Traced Agent

```
def run_agent_traced(user_query, max_iterations=10):
    """Same agent, but collects detailed trace data."""
    # (Same setup as run_agent...)
    trace = []
    token_log = []

    for i in range(max_iterations):
        response = client.chat.completions.create(...)
        tokens_in = response.usage.prompt_tokens
        tokens_out = response.usage.completion_tokens
        token_log.append({"iter": i+1, "in": tokens_in, "out": tokens_out})
        # (Same parsing logic...)

    return trace, token_log
```

📊 Real Token Growth Output

```
trace, tokens = run_agent_traced(
    "Who lived longer – Albert Einstein or Isaac Newton? By how many years?"
)
```

```
=====
TOKEN USAGE PER ITERATION
=====
```

Iter 1:	266 in +	64 out =	330		██████
Iter 2:	355 in +	53 out =	408		████████
Iter 3:	432 in +	50 out =	482		██████████
Iter 4:	507 in +	49 out =	556		███████████
Iter 5:	580 in +	216 out =	796		██████████████

TOTAL: 2,572 tokens
→ prompt_tokens GROWS each iteration – the context engineering problem.

[!WARNING] **Notice the pattern:** 266 → 355 → 432 → 507 → 580. The `prompt_tokens` grow with every iteration because the entire conversation history (system prompt + user message + all previous assistant + observation messages) is re-sent. Over 50+ iterations, this compounds to **hundreds of thousands of tokens** — and that's exactly why context management strategies (RAG, compression summaries) are essential.

8 Level Up: The Code Agent

Give the agent **file I/O + Python execution**. These are essentially the same tools Claude Code has:

The Coding Toolset

Tool	Signature	Purpose
<code>read_file</code>	<code>read_file(path: str)</code>	Read a file and return its contents
<code>write_file</code>	<code>write_file(path: str, content: str)</code>	Write content to a file
<code>run_python</code>	<code>run_python(code: str)</code>	Execute Python code, return stdout/stderr
<code>list_files</code>	<code>list_files(path: str)</code>	List files in a directory
<code>calculate</code>	<code>calculate(expression: str)</code>	Evaluate a math expression

```
def read_file(path):
    """Read a file's contents."""
    try:
        with open(path, "r") as f:
            content = f.read()
            return json.dumps({"path": path, "content": content[:3000], "truncated":
len(content) > 3000})
    except Exception as e:
        return json.dumps({"error": str(e)})

def write_file(path, content):
    """Write content to a file."""
    try:
        d = os.path.dirname(path)
        if d: os.makedirs(d, exist_ok=True)
        with open(path, "w") as f:
            f.write(content)
        return json.dumps({"status": "success", "path": path, "bytes":
len(content)})
    except Exception as e:
        return json.dumps({"error": str(e)})

def run_python(code):
    """Execute Python code and return stdout/stderr."""
    try:
        result = subprocess.run(
            ["python3", "-c", code],
            capture_output=True, text=True, timeout=15,
```



```

    )
    return json.dumps({
        "stdout": result.stdout[:2000] if result.stdout else "",
        "stderr": result.stderr[:2000] if result.stderr else "",
        "exit_code": result.returncode,
    })
except subprocess.TimeoutExpired:
    return json.dumps({"error": "Timed out (15s limit)"})

def list_files(path="."):
    """List files in a directory."""
    try:
        items = sorted(os.listdir(path))
        return json.dumps({"path": path, "files": items[:50]})
    except Exception as e:
        return json.dumps({"error": str(e)})

```

The Code Agent's Enhanced System Prompt

```
CODE_SYSTEM_PROMPT = """You are a coding agent. You write, run, and debug Python code.
```

```
Available tools: {tool_descriptions}
```

```
## Rules
```

- ONE action per turn. Wait for OBSERVATION.
 - After writing code to a file, always run_python to TEST it.
 - If a test fails, read the error, fix the code, try again.
 - Include print() statements so you can see output.
- ```
"""
```

### Test: Write, Run, and Verify Code

```

result = run_code_agent(
 "Write a Python function called 'is_prime(n)' that checks if a number is prime. "
 "Save it to 'prime.py'. Then write a test script that imports it and tests with: "
 "1, 2, 13, 15, 97, 100. Print the results."
)

```

```

=====
🧑 TASK: Write 'is_prime(n)' function...
=====

--- Iteration 1/15 ---
🧠 THOUGHT: I'll write the is_prime function to prime.py...

```

```
=====
[✓] DONE in 1 iteration(s)
=====
📄 The function `is_prime(n)` has been implemented in `prime.py`:

1: False
2: True
13: True
15: False
97: True
100: False
```

🔧 Test: Data Task (CSV + Report)

```
result = run_code_agent(
 "Create 'employees.csv' with columns: name, department, salary. "
 "Add 10+ rows across Engineering, Sales, Marketing. "
 "Then write a script that calculates average salary per department."
)
```

```
--- Iteration 1/15 ---
💭 THOUGHT: I'll create a script that generates the CSV with realistic data...
🔑 ACTION: run_python
🕒 OBSERVATION: {"stdout": "employees.csv created successfully with 12 rows\n"}

--- Iteration 2/15 ---
💭 THOUGHT: Now I'll write a script to read the CSV and calculate averages...
🔑 ACTION: run_python
🕒 OBSERVATION: {"stdout": "\nEMPLOYEE SALARY REPORT\n... Engineering 4 $455000 $113750 ..."}

--- Iteration 3/15 ---
[✓] DONE in 3 iteration(s)
```

[!NOTE] **Watch the pattern:** The code agent follows the same ReAct loop — it just has different tools. Instead of searching Wikipedia, it's writing files, running code, and debugging errors autonomously.

## 9 Native Function Calling vs. String Parsing

Our Part 3 agent uses **string parsing** (regex on **THOUGHT/ACTION**). Modern APIs support **native function calling** — the model outputs structured **tool\_calls** that don't need regex.

### The Difference

| Approach | How Tools Are Called | Parsing | Production Use |
|----------|----------------------|---------|----------------|
|----------|----------------------|---------|----------------|

| Approach                  | How Tools Are Called                                     | Parsing                                  | Production Use             |
|---------------------------|----------------------------------------------------------|------------------------------------------|----------------------------|
| String Parsing<br>(ReAct) | Agent outputs <b>ACTION:</b><br><b>tool_name</b> as text | Regex ( <b>re.search</b> )               | Educational, full control  |
| Native Function Calling   | API returns structured<br><b>tool_calls</b> object       | <b>json.loads()</b> on structured output | Production-grade, reliable |

Native Agent Implementation

```
def run_native_agent(user_query, max_iterations=10):
 """Agent using native function calling. No regex – the API returns structured tool_calls."""
 messages = [
 {"role": "system", "content": "You are a helpful research assistant. Use tools to find information."},
 {"role": "user", "content": user_query},
]

 for i in range(max_iterations):
 response = client.chat.completions.create(
 model=TOOL_MODEL, messages=messages,
 tools=NATIVE_TOOLS, temperature=0, max_tokens=800,
)
 msg = response.choices[0].message
 finish = response.choices[0].finish_reason

 # Text response = done
 if finish == "stop" and msg.content:
 return {"answer": msg.content, "iterations": i + 1}

 # Tool calls
 if msg.tool_calls:
 messages.append(msg) # Append the full message with tool_calls
 for tc in msg.tool_calls:
 fn_name = tc.function.name
 fn_args = json.loads(tc.function.arguments) if
tc.function.arguments else {}
 result = NATIVE_FNS[fn_name](**fn_args) if fn_name in NATIVE_FNS
else json.dumps({"error": f"Unknown tool: {fn_name}"})
 messages.append({"role": "tool", "tool_call_id": tc.id, "content":
result})

 return {"answer": "Max iterations reached", "iterations": max_iterations}
```

result = run\_native\_agent("What's the date today and what will be tomorrow?")

```
--- Iteration 1 ---
🔑 TOOL: get_current_date({})
🕒 RESULT: {"datetime": "2026-03-07 13:01:52"}

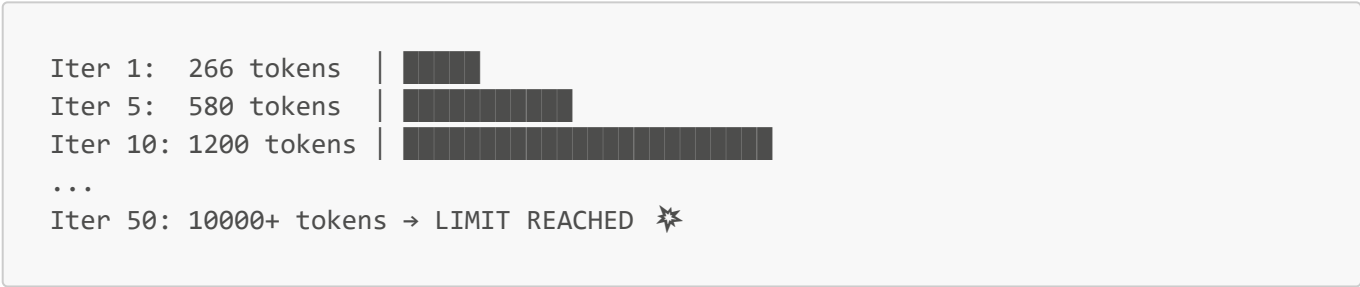
--- Iteration 2 ---
✅ FINAL ANSWER:
 The current date is **March 7, 2026**. Tomorrow will be **March 8, 2026**.
```

[!TIP] **Key advantage of native function calling:** No regex fragility. The API guarantees structured output, eliminating the risk of the model formatting its **ACTION:** line slightly wrong and breaking your parser.

## 10 The "Gotchas" of Autonomous Agents

### 🧠 Memory Management (The Context Window Problem)

Agents operate within a finite Context Window. If an agent runs for 50 iterations, the token count explodes:



#### How do we fix this?

| Strategy               | How It Works                                                            | Trade-off                                   |
|------------------------|-------------------------------------------------------------------------|---------------------------------------------|
| Aggressive Compression | Run a smaller, cheap LLM to summarize past turns into dense paragraphs  | Lossy — may lose critical details           |
| RAG (Retrieval)        | Store observations in a vector DB; retrieve only relevant ones per turn | Adds latency and infrastructure complexity  |
| Sliding Window         | Keep only the last N messages + the system prompt                       | Simple, but agent "forgets" early context   |
| Structured Memory      | Maintain a running JSON "scratchpad" that the agent updates each turn   | More tokens per turn, but controlled growth |

### 🧠 Hallucinations = The Infinite ReAct Loop

When a normal chatbot hallucinates, it gives you a bad answer. When an **agent** hallucinates, it enters a **death spiral**:

| Failure Mode | What Happens | Example |
|--------------|--------------|---------|
|--------------|--------------|---------|

| Failure Mode    | What Happens                                             | Example                                                                                     |
|-----------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Phantom Tools   | Agent calls a tool that doesn't exist                    | Repeatedly calling <code>read_dir()</code> when the tool is named <code>list_files()</code> |
| Phantom Context | Agent assumes a variable/file exists without checking    | Writing a fix for an imaginary bug, polluting its own context                               |
| Goal Drift      | Agent gets distracted by a side-error                    | You asked for feature X; it spent 20 iterations debugging an import typo in file Y          |
| Infinite Retry  | Tool returns an error; agent retries the exact same call | <code>search_wikipedia("Einstein")</code> fails 10 times in a row with no strategy change   |

[!CAUTION] The `max_iterations` Parameter is **NOT Optional**. Without it, a hallucinating agent will loop forever, burning tokens and money. Always set a hard ceiling: `max_iterations=10` for simple tasks, `max_iterations=25` for complex multi-step workflows.

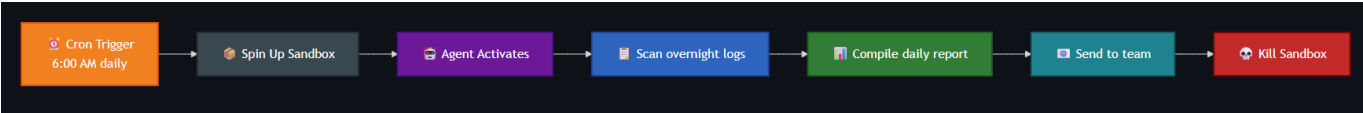
⚡ Why `Tokens/Second` (tk/s) is the Most Critical Agent Metric

| Scenario                       | Turns | At 50 tk/s     | At 200 tk/s |
|--------------------------------|-------|----------------|-------------|
| Simple chat                    | 1     | 2s             | 0.5s        |
| ReAct agent (simple task)      | 5     | 10s            | 2.5s        |
| Code agent (write + test loop) | 15    | 30s            | 7.5s        |
| Complex multi-file refactor    | 50    | 100s (1.7 min) | 25s         |

In standard chat, you wait for ONE response. In a ReAct loop, the model makes 5-50 **hidden** autonomous turns before showing you the result. If your model runs at slow `tk/s`, the agent feels **cripplingly sluggish**. Speed is paramount.

🤖 Autonomous Schedulers (Cron Jobs)

Modern agents aren't just one-off executions — they are **continuous**:



The agent remains **entirely dormant** until its execution loop is triggered by the environment. It wakes, works, reports, and dies — all without human intervention.

🛠️ Homework / Challenges

To solidify these concepts, modify the Colab notebook with these challenges:

| # | Challenge | What You'll Learn |
|---|-----------|-------------------|
|---|-----------|-------------------|

| # | Challenge                                                                                                                                                                                             | What You'll Learn                           |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1 | <b>Context Management</b> — After every 5 iterations, compress <code>messages</code> via a smaller LLM call. Compare token usage before/after.                                                        | The Context Engineering Problem in practice |
| 2 | <b>Planning Step</b> — Before the ReAct loop, have the LLM output a concrete <code>PLAN</code> (list of steps). Force the agent to follow each step sequentially. Compare Plan→Execute vs pure ReAct. | Plan-and-Execute architecture               |
| 3 | <b>Multi-Agent</b> — Create a 'researcher' (Wikipedia tools) and a 'coder' (file/code tools). Build an orchestrator that routes subtasks to the right agent.                                          | Multi-agent systems                         |
| 4 | <b>Web Browsing</b> — Add a <code>fetch_webpage(url)</code> tool using <code>requests</code> + <code>BeautifulSoup</code> . Have the agent follow links. What new failure modes appear?               | Real-world tool fragility                   |

Made with  by [Akshat Shethia](#)